Somaiya Vidyavihar University
K. J. Somaiya College of Engineering

Department of Computer Engineering · L.Y. B.Tech · Semester VII (2026-2027)

Course: **DevOps** (216U01E744) · **IA-2 Case Study Assignment**

Monitoring and Data Visualization using Grafana

Tool: **Grafana Tempo** (with Grafana, Prometheus and OpenTelemetry)

Group member	Roll no.
Amrit Nigam	16010123038
Omik Acharya	16010123218
Amandeep Singh	16010123036

Faculty in-charge: **Mr. Swapnil Pawar (SCP)**

Date: October 2026

Abstract

Modern applications are split into many small services, so when something goes wrong it is hard to tell *which* service is at fault. This case study builds a complete, reproducible observability stack that answers both questions an operator asks during an incident: *is something wrong?* and *where is it wrong?* Prometheus collects system metrics (CPU, memory, disk, network via Node Exporter) and application RED metrics from two instrumented Python microservices; Grafana Tempo stores the distributed traces those services emit through OpenTelemetry; Grafana visualises both on three dashboards and evaluates six alert rules that send email notifications. Exemplars and Tempo's metrics-generator link the two worlds, so a latency spike on a graph opens the exact slow trace in one click. The whole stack — services, data sources, dashboards, alert rules and contact points — is defined as code and starts with a single `docker compose up`. We demonstrate it with four fault-injection scenarios and measure its footprint and alerting delay.

1. Introduction

Monitoring watches known indicators against known thresholds ("is CPU above 80 %?"). **Observability** is the broader ability to understand a system's internal state from its outputs, including questions nobody predicted. It rests on three pillars:

Pillar	What it is	Strength	Tool in this project
Metrics	Numeric time series aggregated over time	Cheap, ideal for dashboards and alerting	Prometheus + Node Exporter
Logs	Discrete, timestamped event records	Detailed context of individual events	stdout logs with `trace_id` (Loki is future work)
Traces	The end-to-end path of one request across services, as a tree of timed *spans*	Shows *where* time is spent and *which* service failed	Grafana Tempo + OpenTelemetry

In the DevOps lifecycle (Plan → Code → Build → Test → Release → Deploy → **Operate** → **Monitor**) observability closes the feedback loop: what is learned while operating the system feeds back into planning and coding. Keeping the monitoring configuration itself in version control ("observability as code") makes it reviewable, reproducible and testable in CI, exactly like application code.

2. Problem Statement and Objectives

Problem. In a microservice system, a dashboard can show that the 95th-percentile latency of the shop went from 200 ms to 1.7 s, but not which of the services involved caused it. Engineers lose time guessing.

Objectives (mirroring the registered practical demonstration, extended with the registered tool):

1. Set up Grafana with Prometheus as a monitoring data source.
2. Build dashboards visualising **CPU, memory, disk, network** and **application performance** with graphs and gauges.
3. Configure **alerts** that fire when a selected metric exceeds a threshold, and deliver them by email.

4. Deploy **Grafana Tempo** as the tracing backend, instrument the services with OpenTelemetry, and show a service map, TraceQL search and a one-click jump from a metric spike to the responsible trace.
5. Provision everything as code, reproducible with one command, and evaluate the result against alternatives with our own measurements.

3. Tool Study

3.1 Grafana

Grafana is an open-source visualisation and alerting platform. Key features used here:

- **Data sources**: 100+ plugins; we use Prometheus (PromQL) and Tempo (TraceQL), cross-linked (exemplars → Tempo, trace → span metrics).
- **Dashboards and panels**: gauge, stat, time series, bar gauge, state timeline, table, node graph, text; template variables (`$service`); dashboard links.
- **Unified alerting**: rules are queries + expressions (reduce, threshold) evaluated on a schedule; *pending* period (`for`) suppresses flapping; notification policies route alerts to **contact points** (email here; Slack, webhook, PagerDuty also possible); multi-dimensional rules produce one alert instance per series.
- **Provisioning**: YAML/JSON files under `/etc/grafana/provisioning` create data sources, dashboards, alert rules, contact points and policies at start-up — no manual clicking.

3.2 Grafana Tempo

Tempo is a high-volume, cost-efficient distributed tracing backend. It does **not** index traces in a database; it writes them to object storage (S3, GCS, Azure or local disk) as columnar **Apache Parquet** blocks.

Component	Role
Distributor	Receives spans (OTLP gRPC/HTTP, Jaeger, Zipkin) and hashes them by trace ID to ingesters
Ingester	Batches spans into blocks in a write-ahead log, then flushes them to storage (`max_block_duration: 5m` here)
Query-frontend / Querier	Splits searches into jobs and reads blocks (lookup by trace ID uses bloom filters)
Compactor	Merges small blocks and enforces retention (`block_retention: 48h`)
Metrics-generator	Derives metrics from spans: `span-metrics` (RED per span), `service-graphs` (edges between services), `local-blocks` (TraceQL metrics); remote-writes them to Prometheus

In this project Tempo runs as a single binary ("monolithic mode"), which contains all components; at scale the same components run as separate microservices.

TraceQL selects spans by attributes, duration, status and structure:

```
{ resource.service.name = "store-api" }                                  # all store-api spans
{ status = error }                                                       # failed spans
{ duration > 500ms }                                                     # slow spans
{ resource.service.name = "inventory-service" && name = "db.query" && duration > 1s }
{ resource.service.name = "store-api" } >> { name = "db.query" }         # structural: descendant spans
```

3.3 Prometheus and Node Exporter

Prometheus **pulls** (scrapes) `/metrics` endpoints on an interval (5 s here) and stores samples in its local time-series database (TSDB). Data is queried with **PromQL**, e.g. CPU utilisation:

```
100 - (avg(rate(node_cpu_seconds_total{mode="idle"}[1m])) * 100)
```

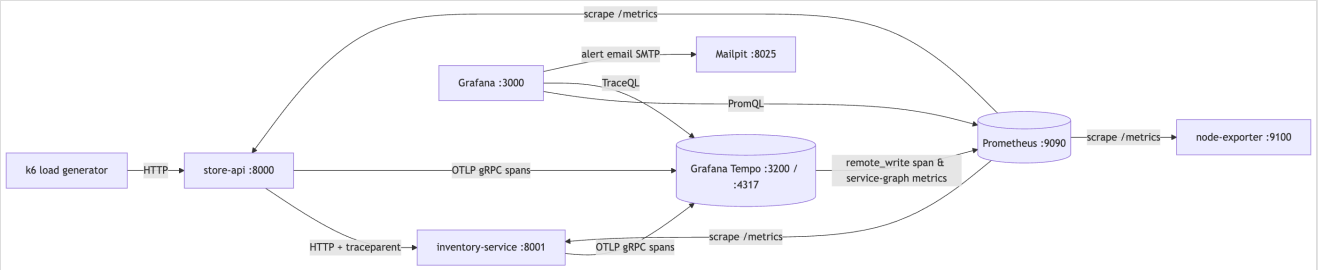
`rate()` turns the ever-increasing idle-seconds counter into "idle seconds per second" (0–1 per core); averaging across cores and subtracting from 100 % gives busy CPU. Node Exporter exposes host metrics (CPU, memory, filesystems, disk I/O, network, load). Prometheus also acts as a **remote-write receiver** for Tempo's generated metrics and stores **exemplars** (sample trace IDs attached to histogram buckets).

3.4 OpenTelemetry

OpenTelemetry (OTel) is the vendor-neutral CNCF standard for telemetry. Our services use the Python SDK:

- A **TracerProvider** with resource attributes (`service.name`, `service.namespace`, `deployment.environment`) and a `BatchSpanProcessor` exporting over **OTLP gRPC** to Tempo.
- **Auto-instrumentation** for Flask (a server span per request) and `requests` (a client span per outgoing call).
- **Context propagation**: the `requests` instrumentation injects the W3C `traceparent` header (`00-<trace-id>-<span-id>-01`) into the call to inventory-service, whose Flask instrumentation extracts it and continues the *same* trace — this is what makes a single trace span both services.
- **Custom spans** (`products.enrich`, `payment.process`, `db.query`, `db.update`) with attributes such as `db.statement`, `payment.method`, `order.amount`.

4. System Architecture



#	Data flow	Path
1	Metrics	apps + Node Exporter expose `/metrics` → Prometheus scrapes every 5 s → Grafana queries with PromQL
2	Traces	each request creates spans → OTLP gRPC → Tempo distributor → ingester → blocks on disk → Grafana queries with TraceQL / trace ID
3	Traces → metrics	Tempo metrics-generator → `traces_spanmetrics_*`, `traces_service_graph_*` → remote write → Prometheus → service map, RED panels
4	Metrics → traces	latency histogram observations carry `trace_id` exemplars → Grafana draws exemplar dots → click opens the trace in Tempo
5	Alerting	Grafana evaluates rules every 10 s → `for` period → notification policy → email contact point → Mailpit

Components and ports: store-api :8000, inventory-service :8001, k6 (≈11 req/s synthetic load), Tempo :3200 / OTLP :4317, Prometheus :9090, Node Exporter :9100 (host network), Grafana :3000, Mailpit (SMTP :1025, inbox :8025); cAdvisor :8081 is optional (`--profile full`).

5. Implementation

5.1 Environment

Item	Version / value
Host	macOS, Docker Desktop 28.3.2 (Linux VM: 10 vCPU, 7.65 GiB RAM), Compose v2.38
Grafana	12.0.2
Grafana Tempo	2.8.1
Prometheus / Node Exporter	v3.4.1 / v1.9.1
k6 / Mailpit	1.0.0 / v1.27
Services	Python 3.12, Flask 3.1, gunicorn 23 (1 worker × 8 threads), prometheus-client 0.22, OpenTelemetry 1.34.1 / 0.55b1

5.2 Repository layout and one-command start

```
docker-compose.yml          all 8 services (+ optional cAdvisor), pinned image tags, named volumes
apps/store-api, apps/inventory-service    app.py, telemetry.py, metrics.py, chaos.py, Dockerfile
prometheus/prometheus.yml   tempo/tempo.yaml   load/k6/script.js
grafana/provisioning/       datasources, dashboard provider, alerting (contact point, policy, 6 rules)
grafana/dashboards/         3 dashboards generated by scripts/gen_dashboards.py
scripts/                    verify.sh, measure.sh, chaos-*.sh
.github/workflows/validate.yml   CI for the monitoring configuration
```

```
cp .env.example .env && docker compose up -d --build && ./scripts/verify.sh
```

`verify.sh` checks 20 things: service health, Tempo/Prometheus readiness, all scrape targets up, app metrics, Tempo-generated span and service-graph metrics, Tempo trace search, Grafana health, provisioned data sources (2), dashboards (3) and alert rules (6), and the Mailpit API.

5.3 Sample application

Two Flask microservices give the traces a real distributed call chain:

- **store-api** — `GET /api/products` calls inventory-service and enriches the result (span `products.enrich`); `POST /api/checkout` reserves stock in inventory-service then processes a payment (span `payment.process`, ~2 % random "payment gateway timeout" → HTTP 502, span status ERROR).
- **inventory-service** — in-memory product "database" (`GET /inventory` with span `db.query`, `POST /reserve` with span `db.update`).
- Both expose `/health`, `/metrics` and **chaos endpoints**: `/chaos/latency`, `/chaos/errors`, `/chaos/cpu` (one busy-loop subprocess per core — subprocesses avoid Python's GIL), `/chaos/reset`, `/chaos/status`.

Instrumentation (`metrics.py`) records, for every request except `/metrics` and `/health`:

Metric	Type	Labels
`http_requests_total`	counter	service, method, route (template, not raw path), status
`http_request_duration_seconds`	histogram (5 ms … 10 s) with `trace_id` exemplar	service, method, route
`http_requests_in_progress`	gauge	service
`orders_total`	counter	result (success/failed)
`inventory_stock_units`	gauge	product
`chaos_active`	gauge	service, type

```
ctx = trace.get_current_span().get_span_context()
trace_id = format(ctx.trace_id, "032x") if ctx.is_valid else None
LATENCY.labels(service_name, request.method, route).observe(
    duration, exemplar={"trace_id": trace_id} if trace_id else None)
```

`/metrics` is served by `prometheus_client.make_wsgi_app()`, which negotiates the **OpenMetrics** format — required for exemplars to reach Prometheus.

5.4 Key configuration

Tempo (`tempo/tempo.yaml`, excerpt):

```
distributor:
  receivers:
    otlp:
      protocols:
        grpc: { endpoint: "0.0.0.0:4317" }
metrics_generator:
  storage:
    remote_write:
      - url: http://prometheus:9090/api/v1/write
        send_exemplars: true
overrides:
  defaults:
    metrics_generator:
      processors: [service-graphs, span-metrics, local-blocks]
```

Prometheus runs with `--web.enable-remote-write-receiver` and `--enable-feature=exemplar-storage`. **Grafana data sources** link Prometheus exemplars to Tempo (`exemplarTraceIdDestinations: trace_id → tempo`) and Tempo to Prometheus for the service map and trace-to-metrics queries.

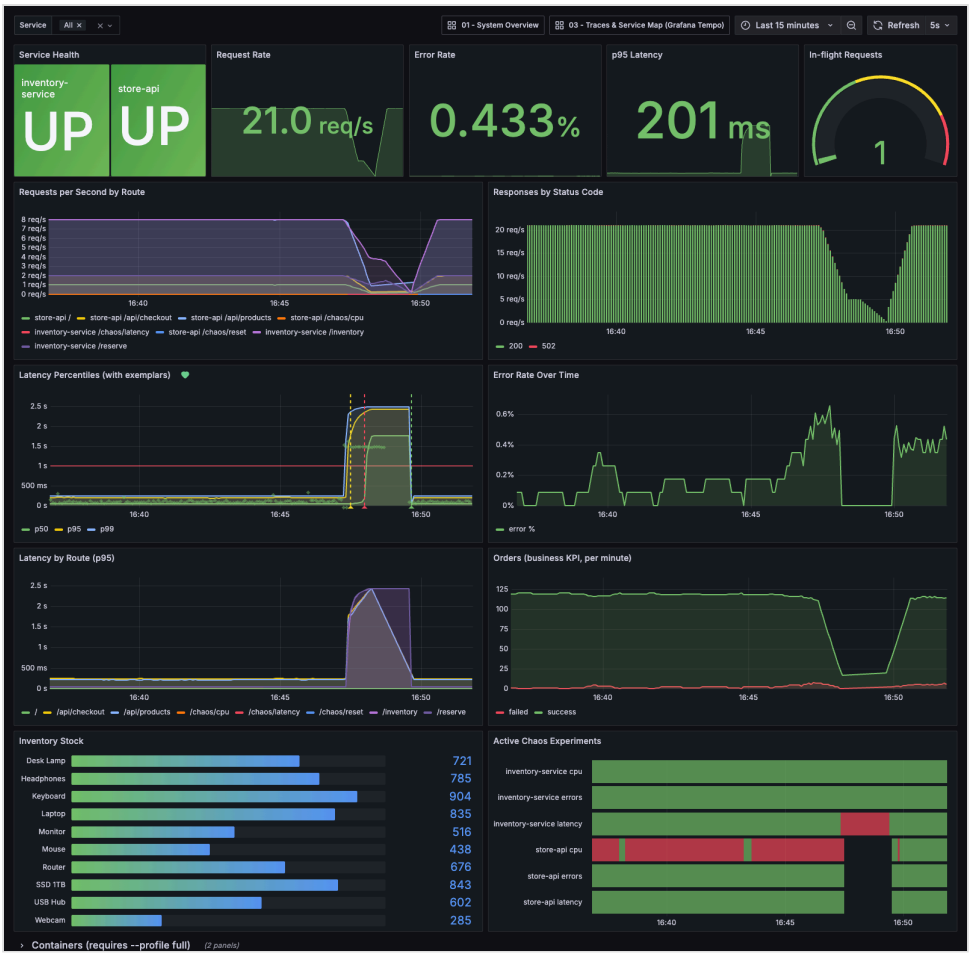
5.5 Dashboards

All three dashboards are generated from one Python script (`scripts/gen_dashboards.py`), refresh every 5 s and link to each other.

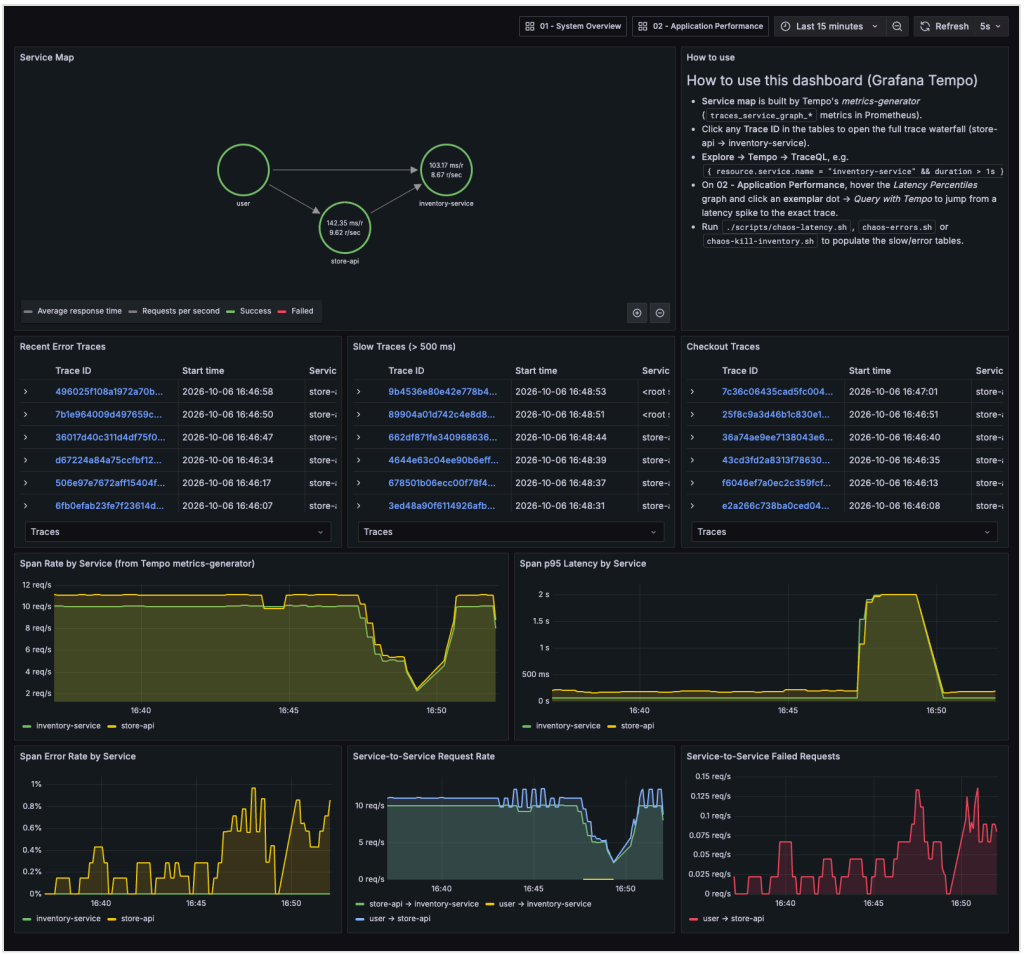
01 – System Overview — CPU, memory and disk gauges (green/yellow/red thresholds), uptime, cores, RAM; CPU by mode with an 80 % threshold line, memory breakdown, network receive/transmit (mirrored), disk I/O, load averages and a filesystem table.



02 – Application Performance — the RED method (Rate, Errors, Duration): service health (UP/DOWN), request rate, error %, p95, in-flight requests; requests by route, responses by status code, p50/p95/p99 latency **with exemplars**, error rate, p95 by route, orders per minute, inventory stock, and a timeline of active chaos experiments.



03 – Traces & Service Map (Grafana Tempo) — Tempo service map, TraceQL tables (recent errors, slow traces > 500 ms, checkout traces), span rate / p95 / error rate per service from the metrics-generator, and service-to-service request and failure rates.



5.6 Alert rules

Six Grafana-managed rules (folder *DevOps IA Alerts*, group `observability-demo`, evaluated every 10 s). Each follows the pattern **A** PromQL query → **B** reduce (last) → **C** threshold. All route to the `devops-team-email` contact point (group wait 10 s, group interval 30 s, repeat 10 min) and send a *Resolved* email on recovery.

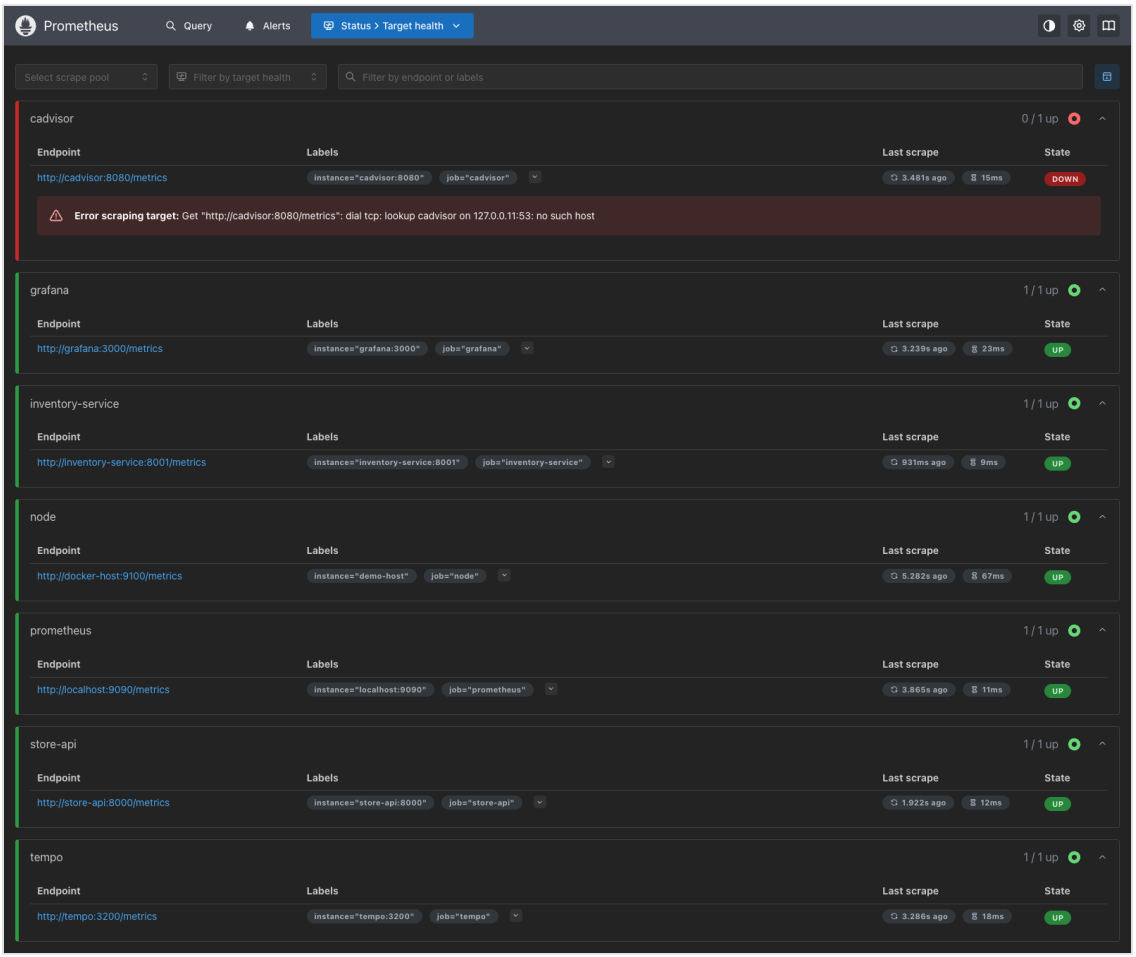
Rule	Expression (A)	Condition	For	No data	Severity
High CPU Usage	`100 - avg(rate(node_cpu_seconds_total{mode="idle"}[1m]))*100`	> 80	30 s	NoData	critical
High Memory Usage	`(1 - MemAvailable/MemTotal) * 100`	> 90	1 m	NoData	warning
Low Disk Space	root-disk used %	> 90	1 m	NoData	warning
High HTTP Error Rate	5xx share of all requests (%)	> 5	30 s	OK	critical
High p95 Latency	`histogram_quantile(0.95, … http_request_duration_seconds_bucket…)`	> 1 s	30 s	OK	warning
Service Down	`up{job=~"store-api	inventory-service"}`	< 1	15 s	Alerting

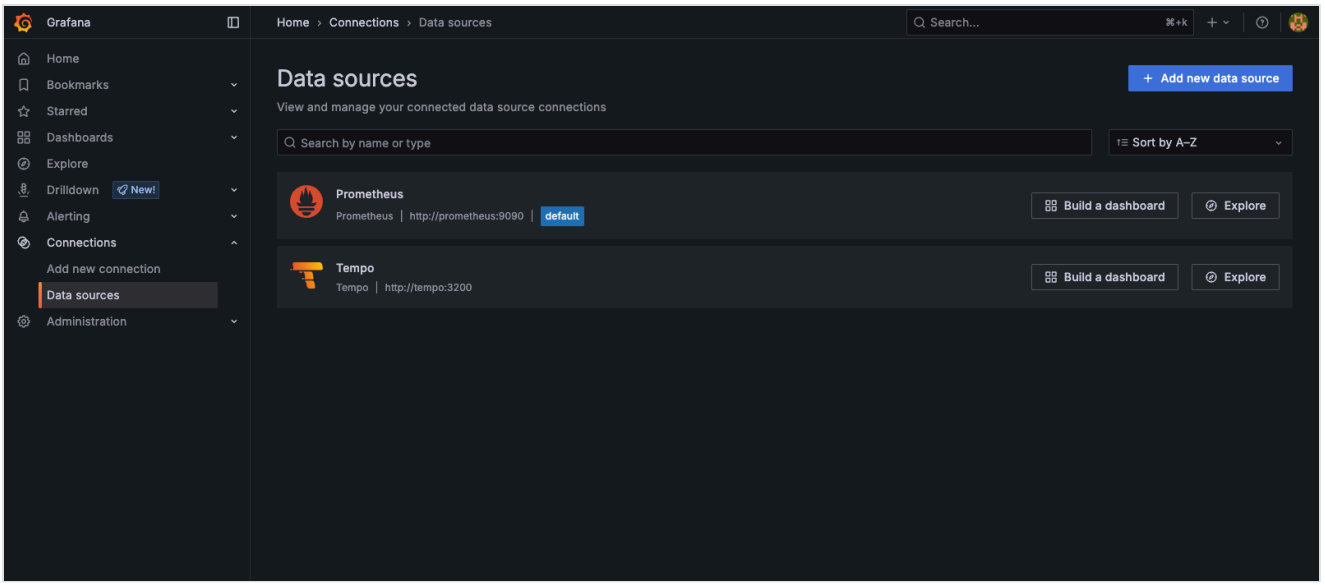
Service Down is **multi-dimensional**: the query returns one series per job, so Grafana creates one alert instance per service and the email names the service that is down.

6. Practical Demonstration Results

6.1 Stack up and provisioned

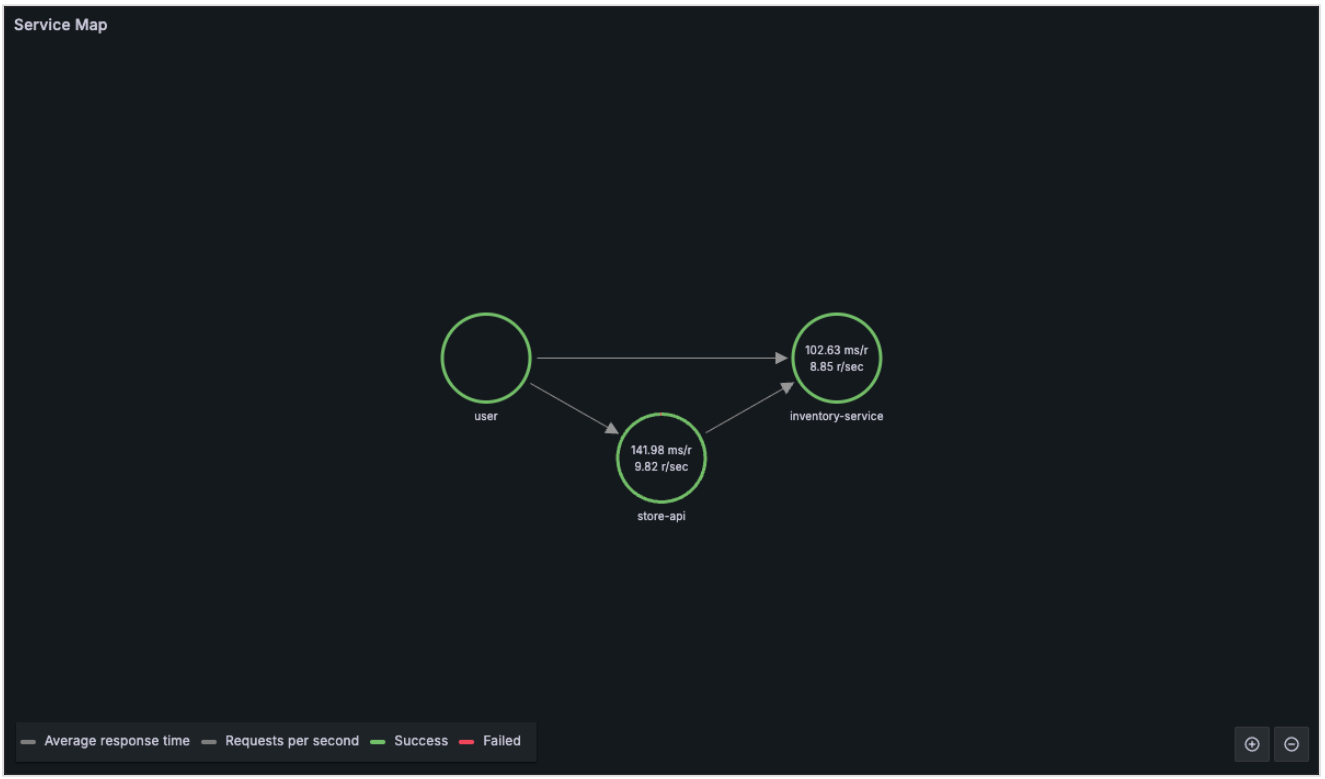
```
$ docker compose ps
NAME                IMAGE                                          STATUS                    PORTS
grafana             grafana/grafana:12.0.2                         Up 2 minutes              0.0.0.0:3000->3000/tcp, [::]:3000->3000/tcp
inventory-service   grafana-tempo-observability-inventory-service  Up 2 minutes (healthy)    0.0.0.0:8001->8001/tcp, [::]:8001->8001/tcp
k6                  grafana/k6:1.0.0                               Up About a minute
mailpit             axllent/mailpit:v1.27                          Up 2 minutes (healthy)    0.0.0.0:1025->1025/tcp, [::]:1025->1025/tcp, 0.0.0.0:8025->8025/tcp, [::]:8025->8025/tcp
node-exporter       prom/node-exporter:v1.9.1                      Up 2 minutes
prometheus          prom/prometheus:v3.4.1                         Up 2 minutes              0.0.0.0:9090->9090/tcp, [::]:9090->9090/tcp
store-api           grafana-tempo-observability-store-api          Up About a minute (healthy)   0.0.0.0:8000->8000/tcp, [::]:8000->8000/tcp
tempo               grafana/tempo:2.8.1                            Up 2 minutes              0.0.0.0:3200->3200/tcp, [::]:3200->3200/tcp, 0.0.0.0:4317-4318->4317-4318/tcp, [::]:4317-4318->4317-4318/tcp
```



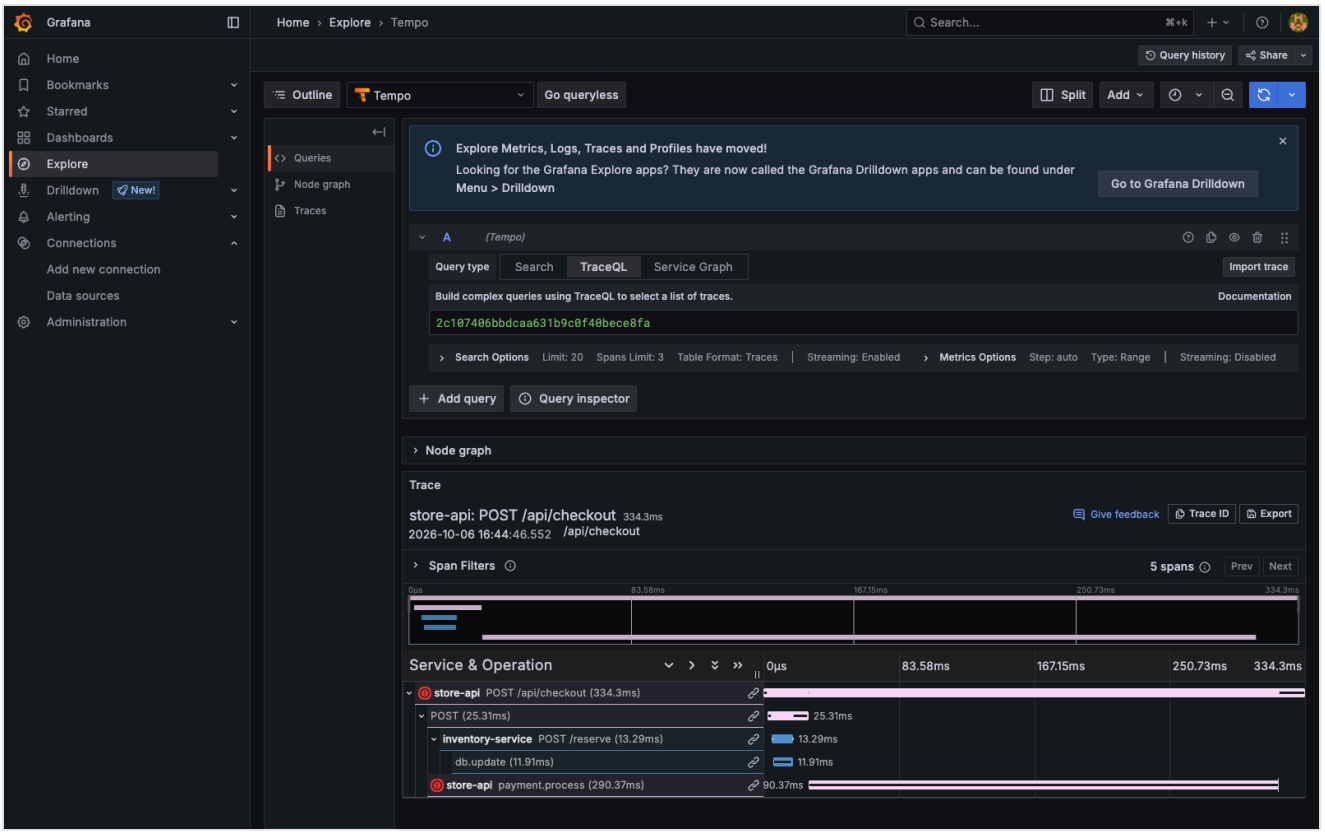


6.2 Distributed tracing with Grafana Tempo

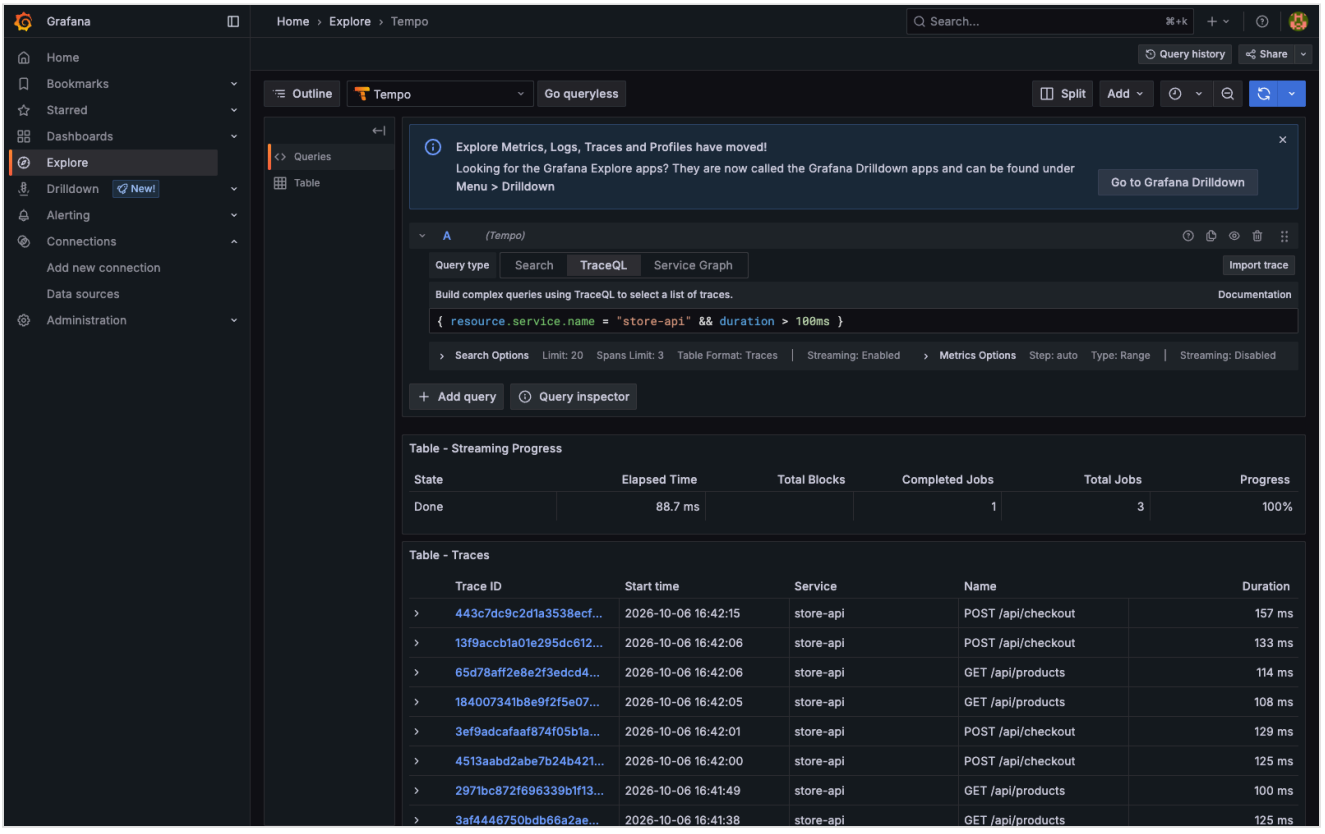
The service map is drawn from Tempo's service-graph metrics: `user` (k6, which is not itself traced) → store-api → inventory-service, with request rate and average latency on each node.



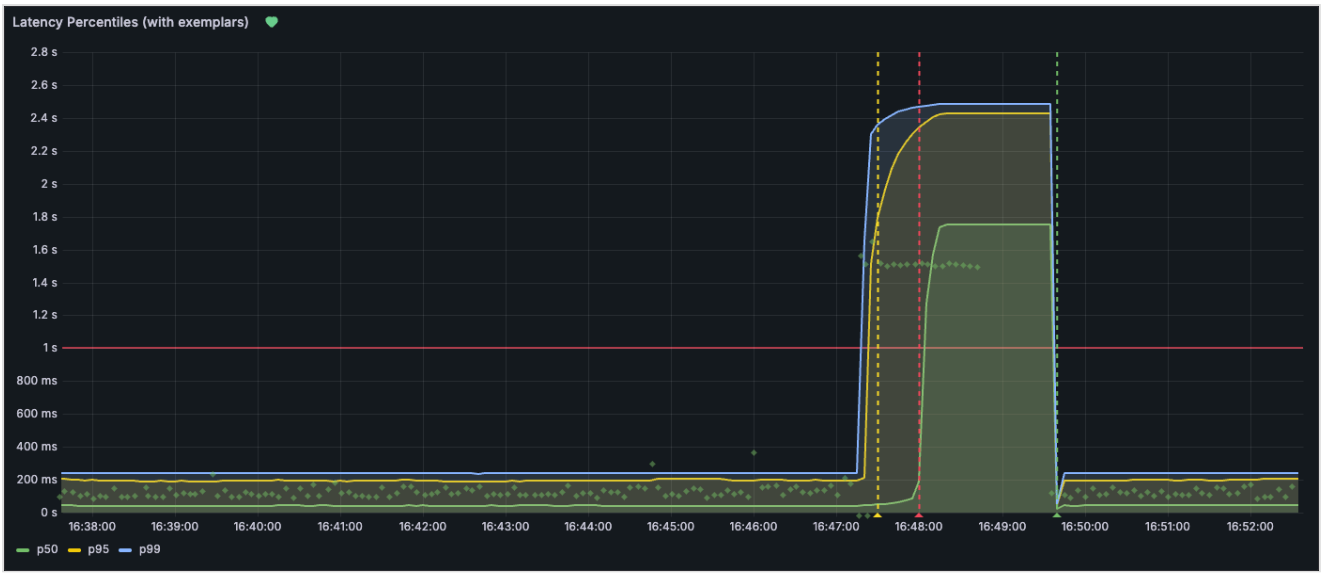
A single checkout trace contains spans from **both** services — proof that the W3C `traceparent` header propagated the context across the HTTP call:



TraceQL search in Explore:

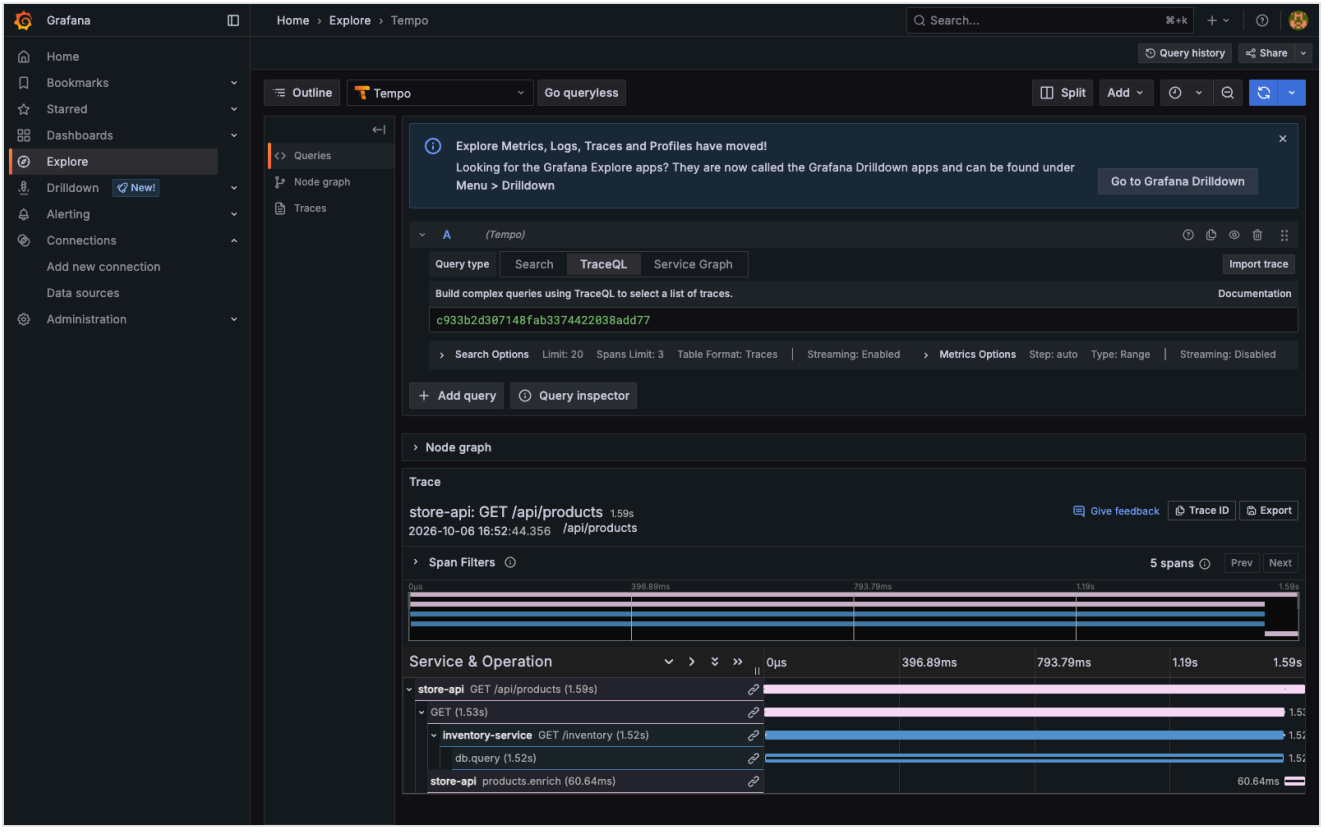


Exemplars (dots) on the latency graph; each carries the `trace_id` of a real request in that latency bucket:



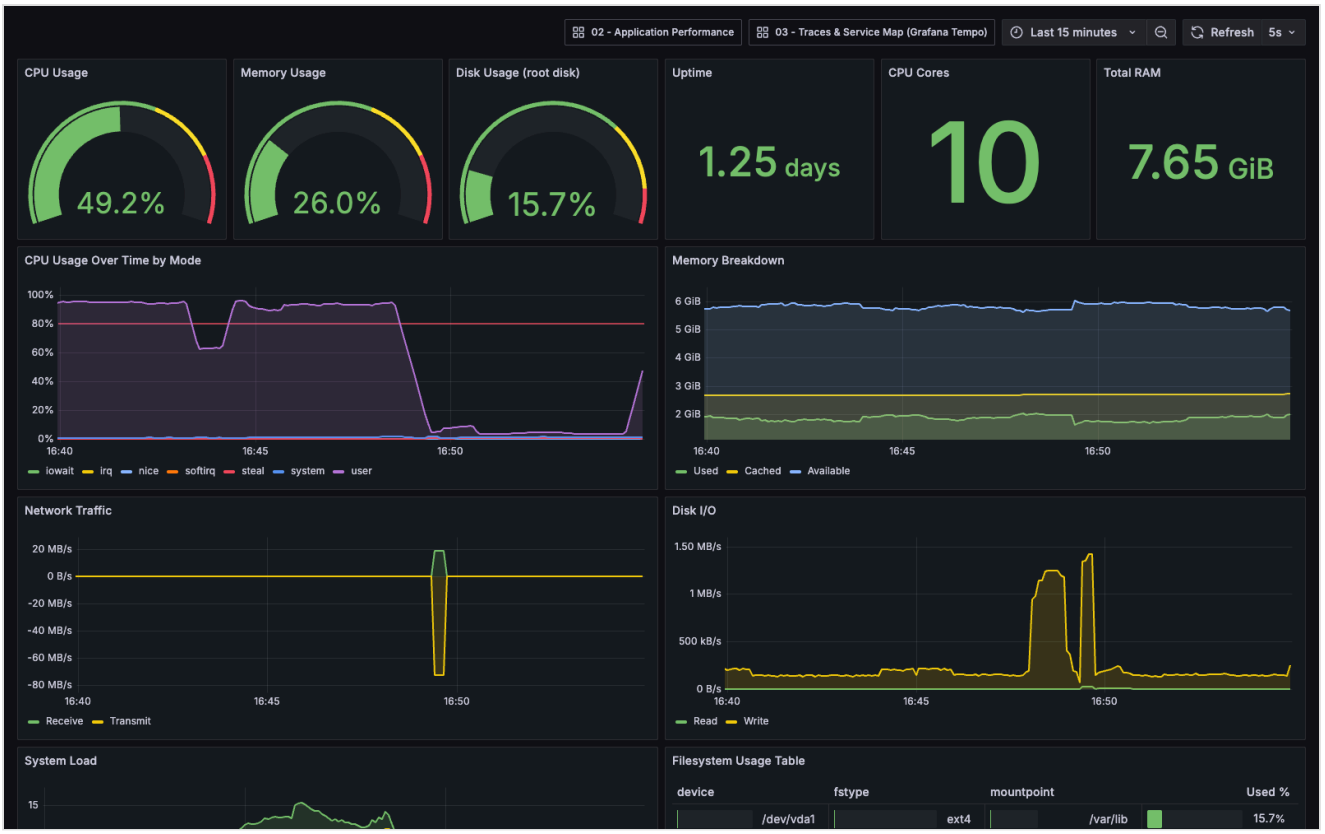
6.3 Scenario 1 — latency in a downstream service (`chaos-latency.sh`)

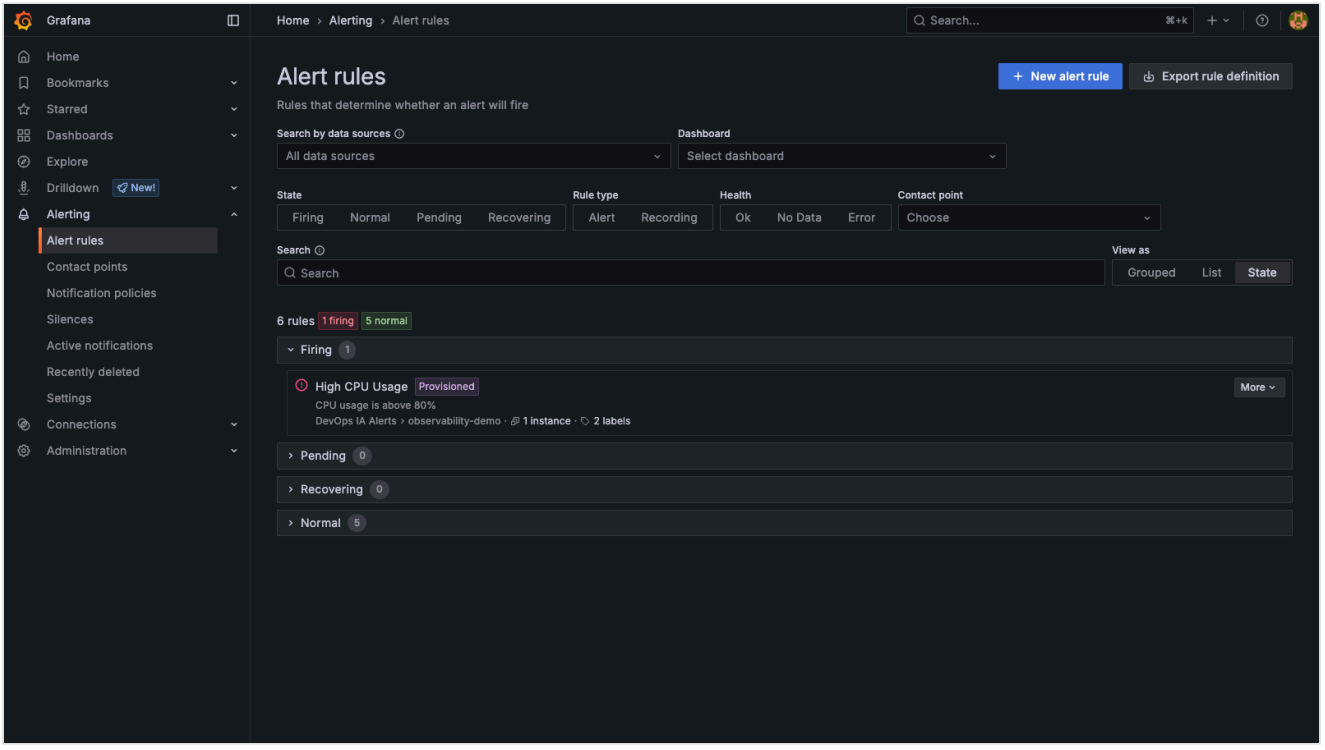
1.5 s of latency was injected into inventory-service. Store-api's p95 crossed 1 s; following an exemplar from the spike leads to a trace where the waterfall immediately shows the slow span is `db.query` **inside inventory-service**, not store-api — root cause in one click instead of guessing.

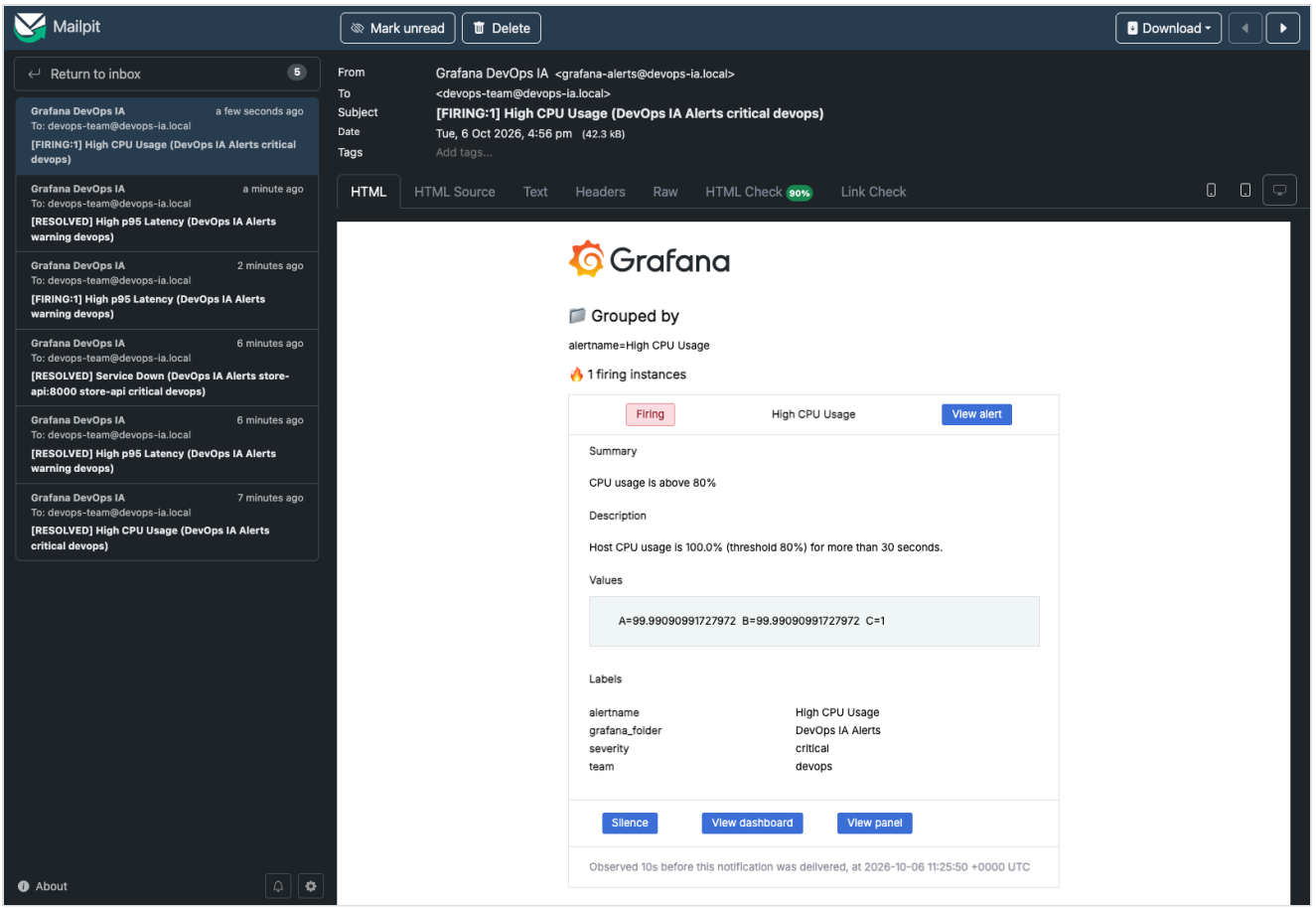


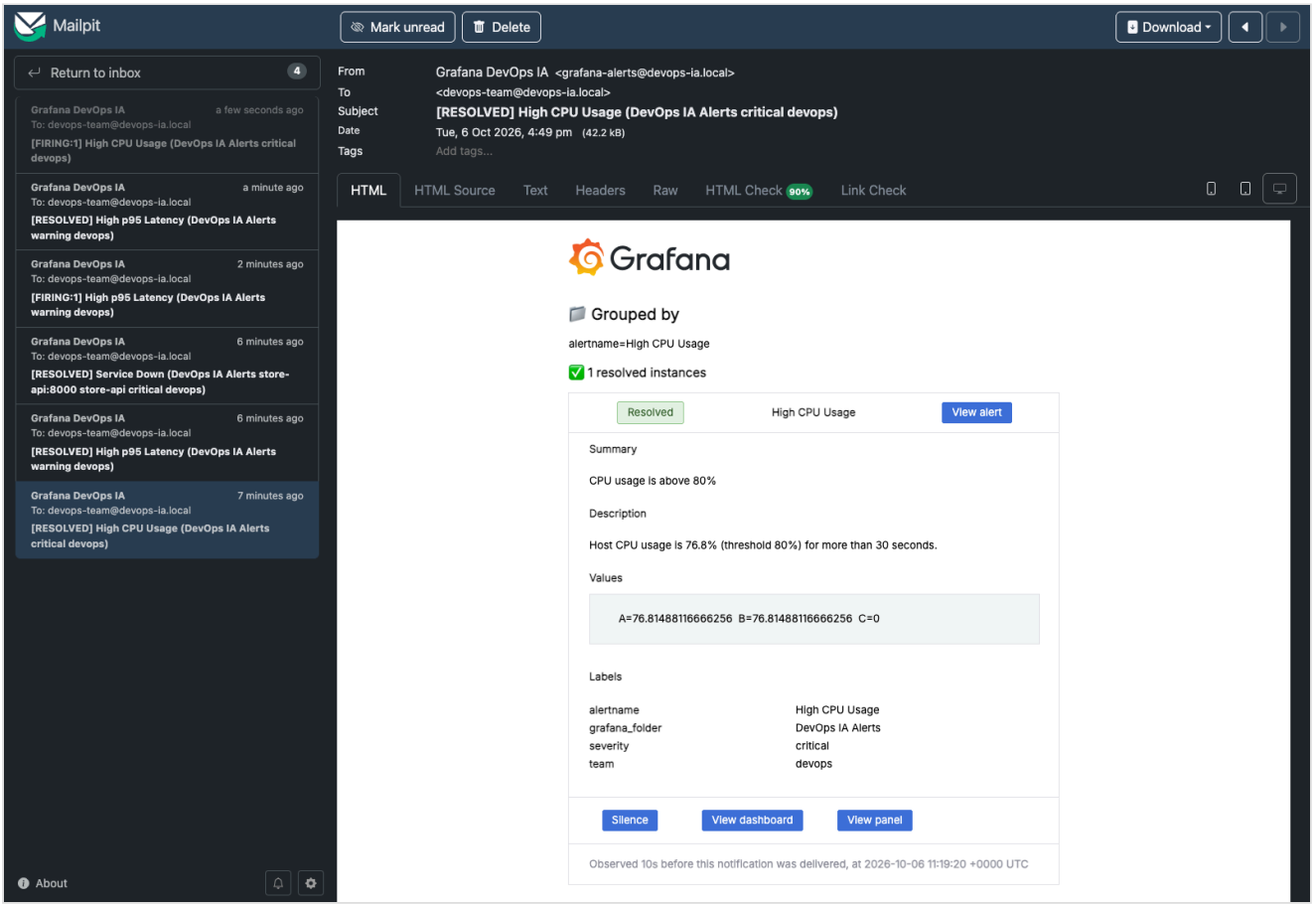
6.4 Scenario 2 — CPU saturation and alert email (`chaos-cpu.sh`)

One busy-loop process per core drives host CPU to ~100 %. The gauge turns red, *High CPU Usage* goes Pending → Firing, the email arrives in Mailpit, and after `chaos-restore.sh` a *Resolved* email follows.


02 - Application Performance
03 - Traces & Service Map (Grafana Tempo)
Last 15 minutes
Refresh 5s
CPU Usage
49.2%
Memory Usage
26.0%
Disk Usage (root disk)
15.7%
Uptime
1.25 days
CPU Cores
10
Total RAM
7.65 GiB
CPU Usage Over Time by Mode
100%
80%
60%
40%
20%
0%
16:40
16:45
16:50
iowait irq nice softirq steal system user
Memory Breakdown
6 GiB
5 GiB
4 GiB
3 GiB
2 GiB
16:40
16:45
16:50
Used Cached Available
Network Traffic
20 MB/s
0 B/s
-20 MB/s
-40 MB/s
-60 MB/s
-80 MB/s
16:40
16:45
16:50
Receive Transmit
Disk I/O
1.50 MB/s
1 MB/s
500 kB/s
0 B/s
16:40
16:45
16:50
Read Write
System Load
15
Filesystem Usage Table
device
fstype
mountpoint
Used %
/dev/vda1
ext4
/var/lib
15.7%

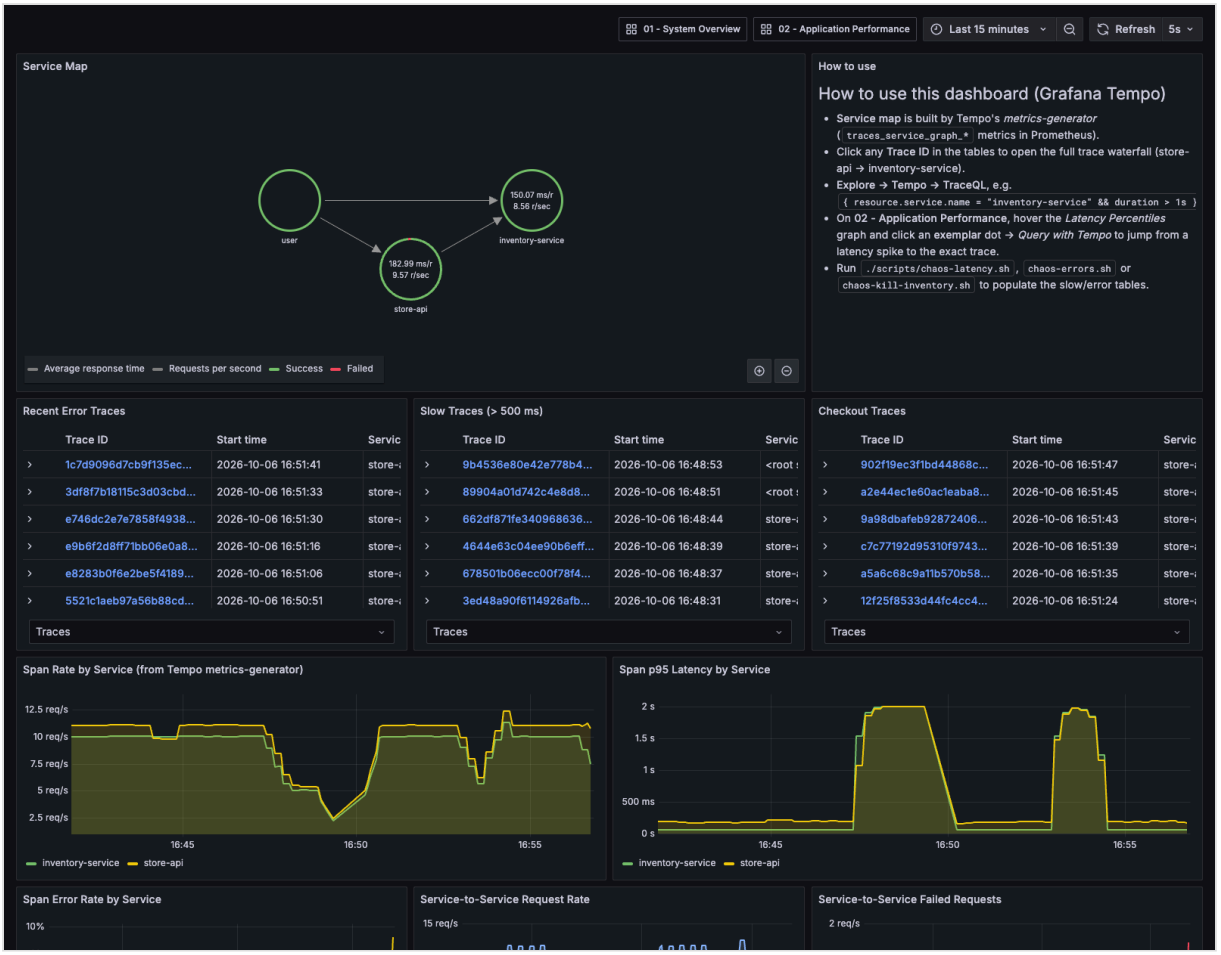

Grafana
Home
Bookmarks
Starred
Dashboards
Explore
Drilldown
New!
Alerting
Alert rules
Contact points
Notification policies
Silences
Active notifications
Recently deleted
Settings
Connections
Administration
Home › Alerting › Alert rules
Search...
Alert rules
Rules that determine whether an alert will fire
New alert rule
Export rule definition
Search by data sources
All data sources
Dashboard
Select dashboard
State
Firing Normal Pending Recovering
Rule type
Alert Recording
Health
Ok No Data Error
Contact point
Choose
Search
Search
View as
Grouped List State
6 rules 1 firing 5 normal
Firing 1
High CPU Usage Provisioned
CPU usage is above 80%
DevOps IA Alerts › observability-demo · 1 instance · 2 labels
More
Pending 0
Recovering 0
Normal 5


Mailpit
Mark unread
Delete
Download
Return to inbox 5
Grafana DevOps IA a few seconds ago
To: devops-team@devops-ia.local
[FIRING:1] High CPU Usage (DevOps IA Alerts critical devops)
Grafana DevOps IA a minute ago
To: devops-team@devops-ia.local
[RESOLVED] High p95 Latency (DevOps IA Alerts warning devops)
Grafana DevOps IA 2 minutes ago
To: devops-team@devops-ia.local
[FIRING:1] High p95 Latency (DevOps IA Alerts warning devops)
Grafana DevOps IA 6 minutes ago
To: devops-team@devops-ia.local
[RESOLVED] Service Down (DevOps IA Alerts store-api:8000 store-api critical devops)
Grafana DevOps IA 6 minutes ago
To: devops-team@devops-ia.local
[RESOLVED] High p95 Latency (DevOps IA Alerts warning devops)
Grafana DevOps IA 7 minutes ago
To: devops-team@devops-ia.local
[RESOLVED] High CPU Usage (DevOps IA Alerts critical devops)
About
From Grafana DevOps IA <grafana-alerts@devops-ia.local>
To <devops-team@devops-ia.local>
Subject [FIRING:1] High CPU Usage (DevOps IA Alerts critical devops)
Date Tue, 6 Oct 2026, 4:56 pm (42.3 kB)
Tags Add tags...
HTML HTML Source Text Headers Raw HTML Check 90% Link Check
Grafana
Grouped by
alertname=High CPU Usage
1 firing instances
Firing High CPU Usage View alert
Summary
CPU usage is above 80%
Description
Host CPU usage is 100.0% (threshold 80%) for more than 30 seconds.
Values
A=99.99090991727972 B=99.99090991727972 C=1
Labels
alertname High CPU Usage
grafana_folder DevOps IA Alerts
severity critical
team devops
Silence View dashboard View panel
Observed 10s before this notification was delivered, at 2026-10-06 11:25:50 +0000 UTC


Mailpit
Mark unread
Delete
Download
Return to inbox 4
Grafana DevOps IA a few seconds ago
To: devops-team@devops-ia.local
[FIRING:1] High CPU Usage (DevOps IA Alerts critical devops)
Grafana DevOps IA a minute ago
To: devops-team@devops-ia.local
[RESOLVED] High p95 Latency (DevOps IA Alerts warning devops)
Grafana DevOps IA 2 minutes ago
To: devops-team@devops-ia.local
[FIRING:1] High p95 Latency (DevOps IA Alerts warning devops)
Grafana DevOps IA 6 minutes ago
To: devops-team@devops-ia.local
[RESOLVED] Service Down (DevOps IA Alerts store-api:8000 store-api critical devops)
Grafana DevOps IA 6 minutes ago
To: devops-team@devops-ia.local
[RESOLVED] High p95 Latency (DevOps IA Alerts warning devops)
Grafana DevOps IA 7 minutes ago
To: devops-team@devops-ia.local
[RESOLVED] High CPU Usage (DevOps IA Alerts critical devops)
About
From Grafana DevOps IA <grafana-alerts@devops-ia.local>
To <devops-team@devops-ia.local>
Subject [RESOLVED] High CPU Usage (DevOps IA Alerts critical devops)
Date Tue, 6 Oct 2026, 4:49 pm (42.2 kB)
Tags Add tags...
HTML HTML Source Text Headers Raw HTML Check 90% Link Check
Grafana
Grouped by
alertname=High CPU Usage
1 resolved instances
Resolved High CPU Usage View alert
Summary
CPU usage is above 80%
Description
Host CPU usage is 76.8% (threshold 80%) for more than 30 seconds.
Values
A=76.81488116666256 B=76.81488116666256 C=0
Labels
alertname High CPU Usage
grafana_folder DevOps IA Alerts
severity critical
team devops
Silence View dashboard View panel
Observed 10s before this notification was delivered, at 2026-10-06 11:19:20 +0000 UTC

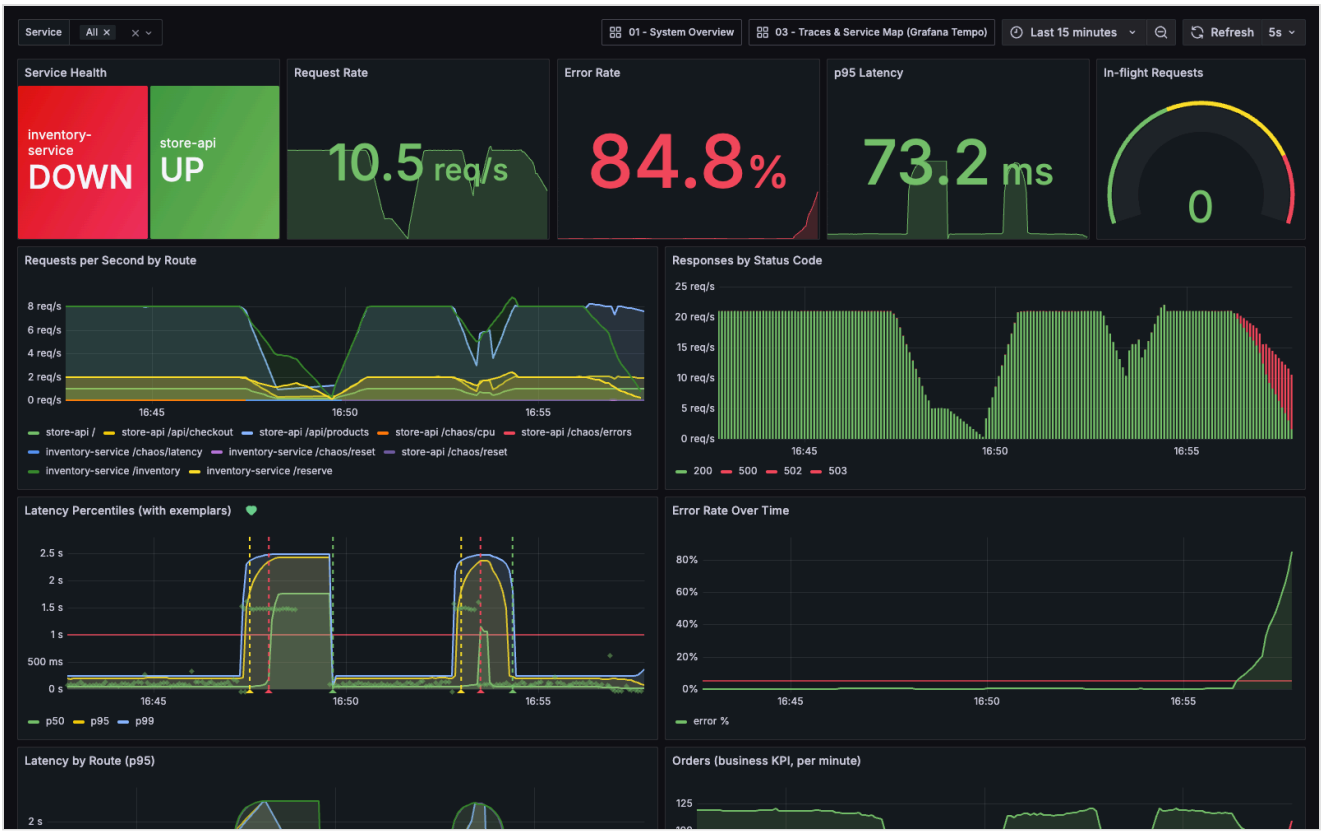
6.5 Scenario 3 — error injection (`chaos-errors.sh`)

40 % of store-api `/api/*` requests return HTTP 500 with the exception recorded on the span. The error-rate panels turn red, *Recent Error Traces* fills with failing traces and *High HTTP Error Rate* fires.

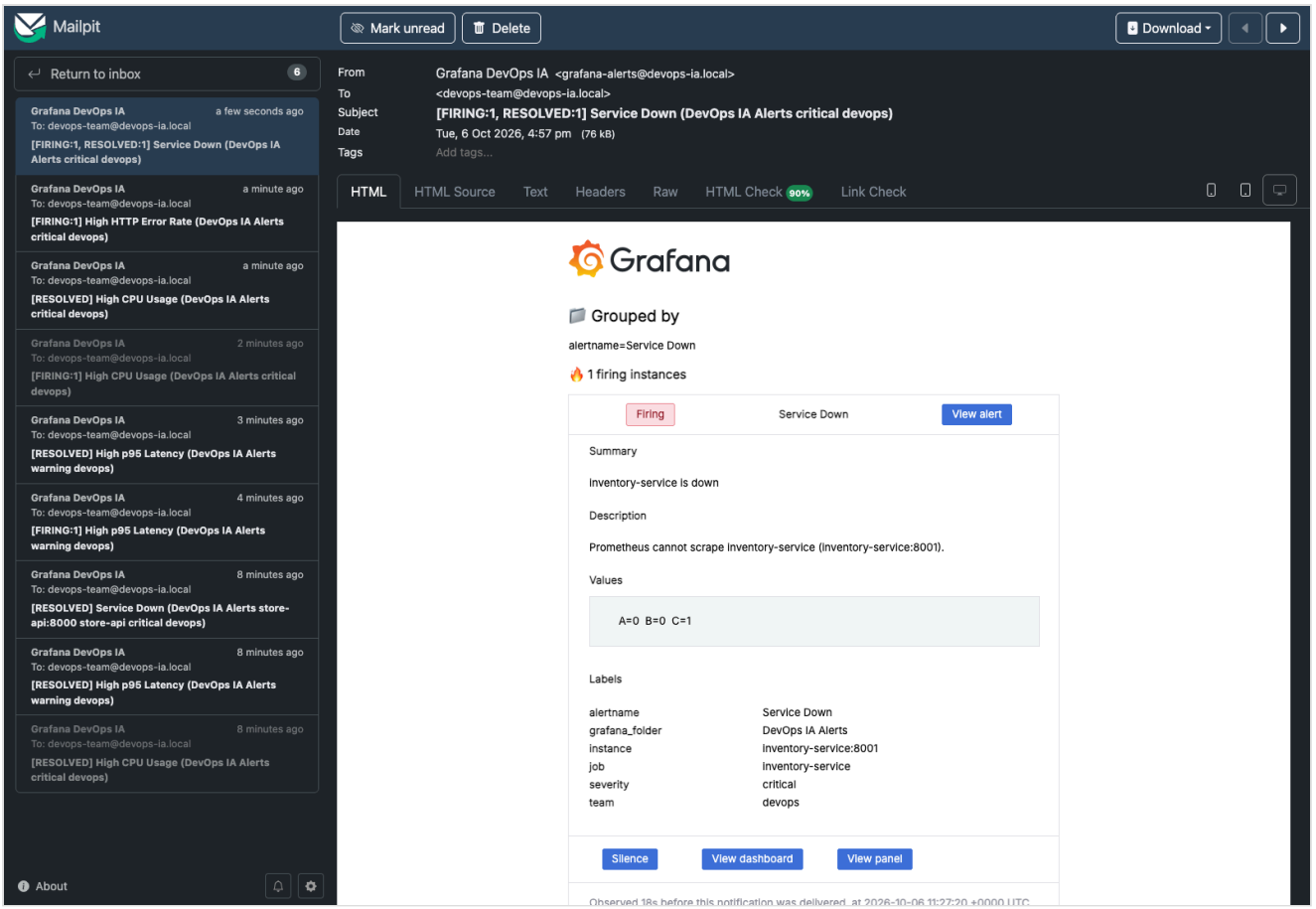


6.6 Scenario 4 — downstream service killed (`chaos-kill-inventory.sh`)

`docker compose stop inventory-service`: the health stat shows DOWN, store-api answers 503 ("inventory unavailable"), failed requests appear on the service map, and *Service Down* fires for `inventory-service` only.


Service
All
01 - System Overview
03 - Traces & Service Map (Grafana Tempo)
Last 15 minutes
Refresh
5s
Service Health
inventory-service
DOWN
store-api
UP
Request Rate
10.5 req/s
Error Rate
84.8%
p95 Latency
73.2 ms
In-flight Requests
0
Requests per Second by Route
8 req/s
6 req/s
4 req/s
2 req/s
0 req/s
16:45
16:50
16:55
store-api /
store-api /api/checkout
store-api /api/products
store-api /chaos/cpu
store-api /chaos/errors
inventory-service /chaos/latency
inventory-service /chaos/reset
store-api /chaos/reset
inventory-service /inventory
inventory-service /reserve
Responses by Status Code
25 req/s
20 req/s
15 req/s
10 req/s
5 req/s
0 req/s
16:45
16:50
16:55
200
500
502
503
Latency Percentiles (with exemplars)
2.5 s
2 s
1.5 s
1 s
500 ms
0 s
16:45
16:50
16:55
p50
p95
p99
Error Rate Over Time
80%
60%
40%
20%
0%
16:45
16:50
16:55
error %
Latency by Route (p95)
2 s
Orders (business KPI, per minute)
125


Service Map
163.50 ms/r
7.95 r/sec
user
inventory-service
181.81 ms/r
9.62 r/sec
store-api
Average response time
Requests per second
Success
Failed


Mailpit
Mark unread
Delete
Download
Return to inbox
6
Grafana DevOps IA a few seconds ago
To: devops-team@devops-ia.local
[FIRING:1, RESOLVED:1] Service Down (DevOps IA Alerts critical devops)
Grafana DevOps IA a minute ago
To: devops-team@devops-ia.local
[FIRING:1] High HTTP Error Rate (DevOps IA Alerts critical devops)
Grafana DevOps IA a minute ago
To: devops-team@devops-ia.local
[RESOLVED] High CPU Usage (DevOps IA Alerts critical devops)
Grafana DevOps IA 2 minutes ago
To: devops-team@devops-ia.local
[FIRING:1] High CPU Usage (DevOps IA Alerts critical devops)
Grafana DevOps IA 3 minutes ago
To: devops-team@devops-ia.local
[RESOLVED] High p95 Latency (DevOps IA Alerts warning devops)
Grafana DevOps IA 4 minutes ago
To: devops-team@devops-ia.local
[FIRING:1] High p95 Latency (DevOps IA Alerts warning devops)
Grafana DevOps IA 8 minutes ago
To: devops-team@devops-ia.local
[RESOLVED] Service Down (DevOps IA Alerts store-api:8000 store-api critical devops)
Grafana DevOps IA 8 minutes ago
To: devops-team@devops-ia.local
[RESOLVED] High p95 Latency (DevOps IA Alerts warning devops)
Grafana DevOps IA 8 minutes ago
To: devops-team@devops-ia.local
[RESOLVED] High CPU Usage (DevOps IA Alerts critical devops)
About
From Grafana DevOps IA <grafana-alerts@devops-ia.local>
To <devops-team@devops-ia.local>
Subject [FIRING:1, RESOLVED:1] Service Down (DevOps IA Alerts critical devops)
Date Tue, 6 Oct 2026, 4:57 pm (76 kB)
Tags Add tags...
HTML
HTML Source
Text
Headers
Raw
HTML Check 90%
Link Check
Grafana
Grouped by
alertname=Service Down
1 firing instances
Firing
Service Down
View alert
Summary
Inventory-service is down
Description
Prometheus cannot scrape inventory-service (inventory-service:8001).
Values
A=0 B=0 C=1
Labels
alertname Service Down
grafana_folder DevOps IA Alerts
instance inventory-service:8001
job inventory-service
severity critical
team devops
Silence
View dashboard
View panel

7. Comparison and Evaluation

7.1 Tracing backends — Grafana Tempo vs alternatives

Criteria	Grafana Tempo	Jaeger	Zipkin	Elastic APM	Datadog APM
License / cost	Open source (AGPLv3); Grafana Cloud paid option	Open source (Apache 2.0), CNCF graduated	Open source (Apache 2.0)	Elastic license / paid tiers	Commercial SaaS, per-host + ingestion pricing
Storage backend	Object storage only (S3, GCS, Azure, local); Parquet blocks	Cassandra, Elasticsearch/OpenSearch, Badger, memory	Cassandra, Elasticsearch, MySQL, memory	Elasticsearch	Vendor-managed
Indexing	No full index → very cheap at scale	Indexed in the DB	Indexed	Indexed	Managed
Query language	TraceQL (attributes, structure, metrics)	Search by service/operation/tags	Search by service/tags	KQL / Kibana UI	Proprietary UI
Ingest protocols	OTLP, Jaeger, Zipkin, OpenCensus	OTLP, Jaeger	Zipkin (OTLP via collector)	OTLP, Elastic agents	Datadog agent, OTLP
Metrics from traces	Built-in metrics-generator (RED + service graph)	SPM with extra components	No	Yes	Yes
UI	Grafana (with metrics, logs, alerts)	Own UI	Own UI	Kibana	Datadog UI
Operational complexity	Low (single binary; microservices mode for scale)	Medium (needs DB)	Low–medium	Medium–high (ES cluster)	Very low (SaaS)
Vendor lock-in	None (OTel)	None	None	Moderate	High
Best fit	Teams on Grafana/Prometheus wanting cheap high-volume tracing	Standalone mature tracing	Simple / legacy setups	Elastic-stack teams	Fully managed all-in-one

7.2 Visualisation / monitoring UIs — Grafana vs alternatives

Criteria	Grafana	Kibana	Prometheus UI	Datadog Dashboards	AWS CloudWatch
Data sources	100+ (Prometheus, Tempo, Loki, SQL, cloud…)	Mainly Elasticsearch	Own TSDB only	Datadog only	AWS metrics/logs
Dashboards	Rich, templated, many panel types	Strong for log analytics	Very basic graphs	Rich	Moderate
Alerting	Unified, multi-channel	Yes (some paid)	Via separate Alertmanager	Yes	Alarms + SNS
Config as code	Provisioning YAML/JSON, Terraform, API	Saved-objects API	Config files	API / Terraform	CloudFormation / Terraform
Cost	Free OSS	Free basic, paid tiers	Free	Paid	Pay-per-use
Lock-in	None	Elastic	None	High	AWS

7.3 Measured results

All values below were measured on our stack with `scripts/measure.sh` (raw output in `docs/measure-output.txt`); none are estimates.

Metric	Value (measured)	How measured
Total RAM used by stack (8 containers)	977 MiB	`docker stats`
Tempo RAM	472 MiB	`docker stats`
Tempo disk after ~37 min (k6 ≈ 11 req/s → ≈ 18 HTTP req/s across both services)	42.6 MB	`du -sh /var/tempo`
Spans ingested per second	≈ 51	`sum(rate(tempo_distributor_spans_received_total[1m]))`
Prometheus head series / disk	9 353 / 14.2 MB	`prometheus_tsdb_head_series`, `du`
Trace lookup by ID (5 traces)	4.5 – 39 ms, avg 13.6 ms	`curl -w %{time_total}` on `/api/traces/<id>`
CPU alert: burn start → Firing	79.3 / 80.0 / 80.0 s, **avg 79.8 s**	`measure.sh`, 3 runs
CPU alert: burn start → email in Mailpit	88.8 / 89.4 / 89.4 s, **avg 89.2 s**	`measure.sh`, 3 runs
Setup time from clone	not measured on a fresh machine yet	manual (fill in after a fresh-clone run)

Alert timing breakdown. Burn-start-to-Firing (≈ 80 s) is longer than the naive "evaluation 10 s + `for` 30 s" because the CPU expression uses `rate(...[1m])`: the 1-minute window must fill with mostly-busy samples before the average crosses 80 % (≈ 30–40 s after the burn starts), then the rule spends 30 s in *Pending*, plus up to 10 s of evaluation-interval alignment and 5 s scrape interval. The email follows ≈ 10 s

after Firing (`group_wait: 10s`). A shorter window (`[30s]`) or `for: 15s` would cut detection time at the cost of more false positives on short spikes — a classic sensitivity vs. noise trade-off.

7.4 Discussion

Strengths observed.

- One pane of glass: metrics, traces and alerts live in Grafana; exemplars and the service map connect them.
- Root-cause time drops from "guess which service" to one click (Scenario 1).
- Everything is provisioned as code and version-controlled; a fresh clone needs one command.
- Tempo is light: it needs only disk/object storage and used under 500 MiB RAM while ingesting ~51 spans/s; trace lookup by ID took 4–39 ms.
- OpenTelemetry keeps the services vendor-neutral — switching to Jaeger or a SaaS would need no code change.

Limitations observed.

- Tempo search relies on recent blocks and TraceQL scans rather than a full index, so very broad historical searches are slower than in an indexed system.
- Local-disk storage is not highly available; production needs S3/GCS and Tempo's microservices mode.
- Grafana alert-rule provisioning YAML is verbose (we generate it).
- On Docker Desktop, Node Exporter measures the Linux VM, not the laptop; the VM has no `/` disk, so the disk queries use `/var/lib/docker` there.
- One gunicorn worker limits app throughput (Prometheus multiprocess mode is needed for more workers).
- The logs pillar (Grafana Loki) is not covered.

8. Challenges Faced and Solutions

Challenge	Cause	Solution
node-exporter failed: `path / is mounted on / but it is not a shared or slave mount`	Docker Desktop rejects `rslave` mount propagation	Mount `/:/host:ro`
Disk gauge showed "No data"	Docker Desktop's VM has no `/` filesystem (only `/var/lib/docker`, virtiofs shares, tmpfs)	Query `mountpoint=~"/`
Network Traffic showed only a few bytes/s	node-exporter in a bridge network sees only its own container's `eth0`	Run node-exporter with `network_mode: host`; pin the compose subnet so Prometheus scrapes it at the bridge gateway (`docker-host:9100`)
`chaos-restore.sh` did not stop the CPU burn	`pkill` is not installed in `python:3.12-slim`	The service tracks its burn subprocesses and terminates them in `/chaos/reset`
`curl :3200/ready` returned "waiting for 15s after being ready"	Tempo's ingester reports ready only 15 s after the first readiness check	`verify.sh` retries the check for up to 30 s
`$__tags` in Tempo → metrics queries disappeared	Grafana provisioning substitutes `$VAR` environment variables	Escape as `$$__tags` in `datasources.yaml`
Measured time-to-alert longer than the configured `for`	`rate([1m])` smoothing (see §7.3)	Documented the breakdown; kept 1 m window to avoid flapping

9. Conclusion and Future Scope

We built a reproducible observability stack that satisfies the registered demonstration — Grafana with Prometheus, CPU/memory/disk/network and application-performance dashboards with gauges and graphs, and threshold alerts delivered by email — and adds Grafana Tempo as a first-class tracing backend. The four chaos scenarios showed the complementary roles: **metrics and alerts tell us *that* something is wrong; traces in Tempo tell us *where and why***, and Grafana connects both.

Future scope: Grafana Loki for logs with trace-to-logs links; Kubernetes deployment with Helm charts; S3/MinIO storage for Tempo; Slack/Teams/Telegram contact points; SLO dashboards with error budgets; an OpenTelemetry Collector or Grafana Alloy pipeline with tail sampling.

10. References

1. Grafana documentation — https://grafana.com/docs/grafana/latest/
2. Grafana Tempo documentation — https://grafana.com/docs/tempo/latest/
3. TraceQL — https://grafana.com/docs/tempo/latest/traceql/
4. Prometheus documentation — https://prometheus.io/docs/
5. Node Exporter — https://github.com/prometheus/node_exporter
6. OpenTelemetry documentation — https://opentelemetry.io/docs/

7. W3C Trace Context — https://www.w3.org/TR/trace-context/
8. k6 documentation — https://grafana.com/docs/k6/latest/
9. Mailpit — https://mailpit.axllent.org/

Appendix A — Commands

```
docker compose up -d --build            # start (add --profile full for cAdvisor)
./scripts/verify.sh                     # 20 health checks
./scripts/chaos-latency.sh 1500 120     # latency in inventory-service
./scripts/chaos-cpu.sh 120              # CPU burn → High CPU Usage alert
./scripts/chaos-errors.sh 0.4 120       # 40 % HTTP 500 → High HTTP Error Rate
./scripts/chaos-kill-inventory.sh       # Service Down
./scripts/chaos-restore.sh              # undo everything
./scripts/measure.sh 3                  # evaluation numbers incl. 3 MTTD runs
docker compose down -v                  # stop and delete data
```

Appendix B — Resource footprint

```
$ docker stats --no-stream
NAME               CPU %     MEM USAGE / LIMIT
grafana            0.34%     173MiB / 7.654GiB
inventory-service  3.72%     56.72MiB / 7.654GiB
k6                 2.99%     38.84MiB / 7.654GiB
mailpit            0.00%     43.63MiB / 7.654GiB
node-exporter      0.00%     16.61MiB / 7.654GiB
prometheus         1.31%     115.2MiB / 7.654GiB
store-api          8.70%     63.71MiB / 7.654GiB
tempo              1.95%     471.9MiB / 7.654GiB
```